Experiment - 5

CRUD operations:

1. Create: db. createCollection (use the " " for some)

2. Insertion:

(i) db. person. insert ({ _id : 1, name : "yashasvini"})

(ii) db. person. insertOne ({ _id : 1, name : "yashasvini", rollno : "23261A6718"})

(iii) db. person. insertMany ({ _id : 1, name : "A", rollno : "A7"},
{_id : 1, name : "B", rollno : "A8"})

3. Read: db. person. find ()

4. Update: db. person. update ({ Actual }, $set : { desired })

db. person. updateOne ({ Actual }, $ set : { desired })

db. person. updateMany ({ Actual }, $ set : { desired })

5. Delete:

db. person. delete ({ Actual }, $ set : { desired })

db. person. deleteOne ({ Actual value })

db. person. deleteMany ({ Actual value })

6. Replace one:

db. person. ({ Actual value }, { desired value })

7. Formatting output:

To see the output in json kind of format

db.person.find().pretty()

MongoDB:

MongoDB is a document database. It stores data in a type of JSON format called BSON. A record in MongoDB is a document, which is data structure composed of key value pairs similar to the structure of JSON objects.

→ Field values may include numbers, strings, booleans, arrays, etc.

Eg:

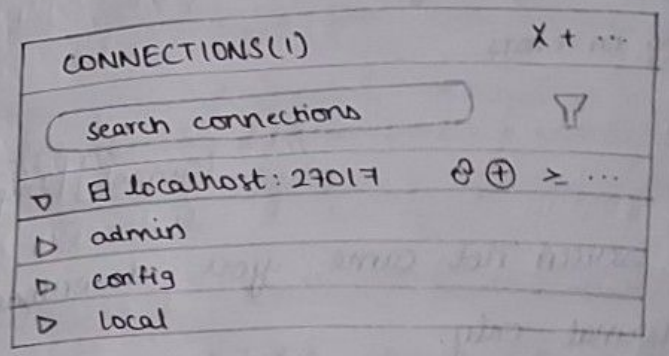
```
{
  title: "Title",
  body: "Body",
  category: "news",
  likes: 1,
  tags: ["news", "events"],
  date: Date()
}
```

Installation:

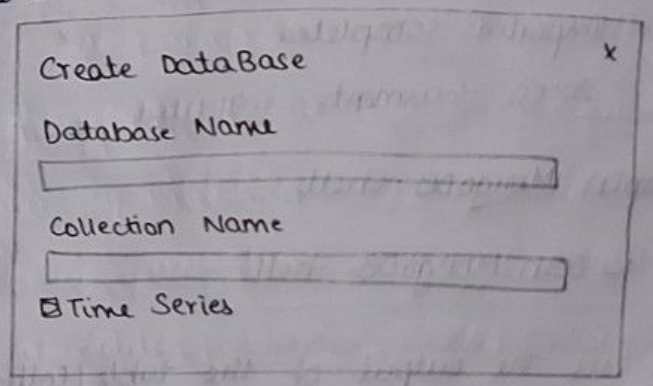
use This link to download the dataset

https://www.kaggle.com/datasets/kundanbedmutha/indian-customer-financial-profiles-and-transactions.

→ Create a "Practice" Database (Click on Plus button)

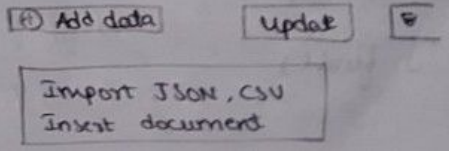


→ Create a "Finance" Collection



→ Down Import the dataset into finance collection

→ Click on "ADD DATA"



→ Now add the data into mongodb

Customer_financial_profiles 08-04-2026

→ Now select all data & import (No changes)

Import ×

To collection Practice_Finance

☑ Ignore empty strings

☐ Stop on errors

Cancel | Import

→ If this screen not came, you downloaded wrong Format only.

✓ Imported completed ×

20000 documents imported

→ Now open MongoDB Shell

> Open MongoDB Shell

→ First see the output of the table/Collection

```
MONGOSH
> use Practice
< Switched to db Practice
> db["Finance"].find()
:
:
```

Tasks!

Basic Exploration & Statistics:

1. Total Document Count: Verify the dataset size.

```
db.customers.countDocuments({})
```

1000

2. Distinct occupations: List all unique job titles.

```
db.customers.distinct("occupation")
```

< ["Engineer", "Doctor", "Teacher", "Business", "Student", "Self Employed"]

3. Average Income: Find the overall average monthly income.

```
db.customers.aggregate([{$group:{_id:null, avgInc:{$avg:"$Income"}}}])
```

[{ "_id": null, "avgInc": 58234.67 }]

4. Income by City: Compare average income across Tier 1, 2, 3 cities.

```
db.customers.aggregate([{$group:{_id:"$City_Tier", avgInc:{$avg:"$Income"}}}])
```

```
[
  { "_id": "Tier 1", "avgInc": 72000},
  { "_id": "Tier 2", "avgInc": 55000},
  { "_id": "Tier 3", "avgInc": 38000}
]
```

5. Top 5 Earners: Find the highest-earning individuals

db.customers.find().sort({Income:-1}).limit(5)

6. Gender Distribution: Count customers by gender.

db.customers.aggregate([{$group:{_id:"$Gender", count:{$sum:1}}}])

7. Age Range Summary: Find the youngest and oldest customers.

db.customers.aggregate([{$group:{_id:null, minAge:{$min:"$Age"}, maxAge:{$max:"$Age"}}}])

8. Monthly Expense Breakdown: Sum total rent paid across all customers.

db.customers.aggregate([{$group:{_id:null, totalRent:{$sum:"$Rent"}}}])

9. Customers with Dependents: Count how many people support 3+ dependents.

db.customers.countDocuments({Dependents:{$gte:3}})

10. Zero Savings Flag: Find customers who have no disposable income left.

db.customers.find({Disposable_Income:{$lte:0}})

Intermediate Behavioral Analysis

1. Identify customers spending > 50% of income on rent.

db.customers.aggregate([{$project:{ratio:{$divide:["$Rent","$Income"]}}}, {$match:{ratio:{$gt:0.5}}}])

2. Average grocery bills per job category.

db.customers.aggregate([{$group:{_id:"$Occupation", avgGroc:{$avg:"$Groceries"}}}])

3. Find users with a 100K+ limit but zero debt.

db.customers.find({credit_limit:{$gt:100000}, total_debt:0})

4. Find customers meeting their "Desired Saving target.

db. customers. find ({ $expr: { $gte: ["$Disposable Income", "$Desired_Savings"]}})

5. Do Tier 1 cities spend more on fun than Tier 3?

db. customers. aggregate ([{ $match: { Age: { $gte 25, $lte: 35 }}}, { $group: { _id: null, avgloan: { $avg: "$Loan_Repayment" }}}])

Advanced Predictive & Risk Analytics:

1. Calculate DTI and flag those above 0.4

db. customers. aggregate ([{ $addFields: { dti: { $divide: ["$total_debt", "$yearly_income"]}}}, { $match: { dti: { $gt: 0.4 }}}])

2. Combine income, credit limit, and savings into one more score.

db. customers. aggregate ([{ $project: { score: { $add: ["$Income", "$credit_limit", "$Disposable_Income"]}}}])

3. High earners with low credit limits.

db. customers. find ({ Income: { $gt: 10000 }, credit_limit: { $lt: 50000 }})

4. Compare current month expenses vs average

db. transactions. aggregate ([{ $group: { _id: { cid: "$client_id", mo: { $month: "$Date" }}, total: { $sum: "$amount" }}}])

Complex Queries:

1. Find customers with high debt relative to their income despite having a decent credit score.

```
db. customers. aggregate([
 {
   $project: {
     id: 1,
     credit_score: 1,
     debt_to_income_ratio: { $divide: [ "$total_debt", "$yearly_income" ]}
   }
 },
 { $match: { debt_to_income_ratio: { $gt: 0.4 },
             credit_score: { $gte: 700 }}},
```

```
{ $ sort : { debt - to - income - ratio: -1}}
])
```

2. Identify the top 5 states by total transaction volume to see where the highest economic activity is occuring.

```
db. transactions. aggregate ([
  {
    $ group: {
      - id: "$ merchant - state",
      total volume: { $ sum; "$ amount"},
      transaction - count: { $ sum : 1}
    }
  },
  { $ sort : { total - volume : -1}},
  { $ limit : 5}
])
```

3. Calculate the average number of credit cards held by different age brackets.

```
db. customers. aggregate([
  {
    $ bucket : {
      groupBy: "$ current - age",
      boundaries : [18, 25, 35, 50, 65, 100],
      default : " other",
      output: {
        avg - cards : { $ avg : "$ num - credit - cards"},
        count : { $ sum : 1}
      }
    }
  }
])
```

4. Flag customers who, if they lost their income, could not survive for more than 2 months based on their current total - savings and Monthly - Expenses.

```
db. customers. aggregate([
  {
    $ addFields: {
      // Burn rate = Total savings / (Rent + Groceries
                                    + Loan Repayment)

      survival - months: {
```

```
$ divide: {
"$total_savings",
{ $add: ["$Rent", "$Groceries", "Loan_Repa
                                -men
]
}
}
},
{
$match: {
survival_months: { $lt: 2},
total_debt: { $gt: 50000}
}
},
{ $sort: { survival_months: 1}}
])
```